

GPUs

CUDA

Vladislav Goncharenko

Senior researcher



MIPT, spring 2026

Recap

Highload

- Foundations
 - von Neumann architecture
 - Flynn's taxonomy
 - Data locality principle
- MapReduce paradigm
 - Hadoop stack
 - YTsaurus
- kNN indexes

Outline

GPUs, CUDA

1. Hardware paradigms
2. Nvidia architectures
3. Contemporary GPUs
4. CUDA infrastructure

Hardware paradigms

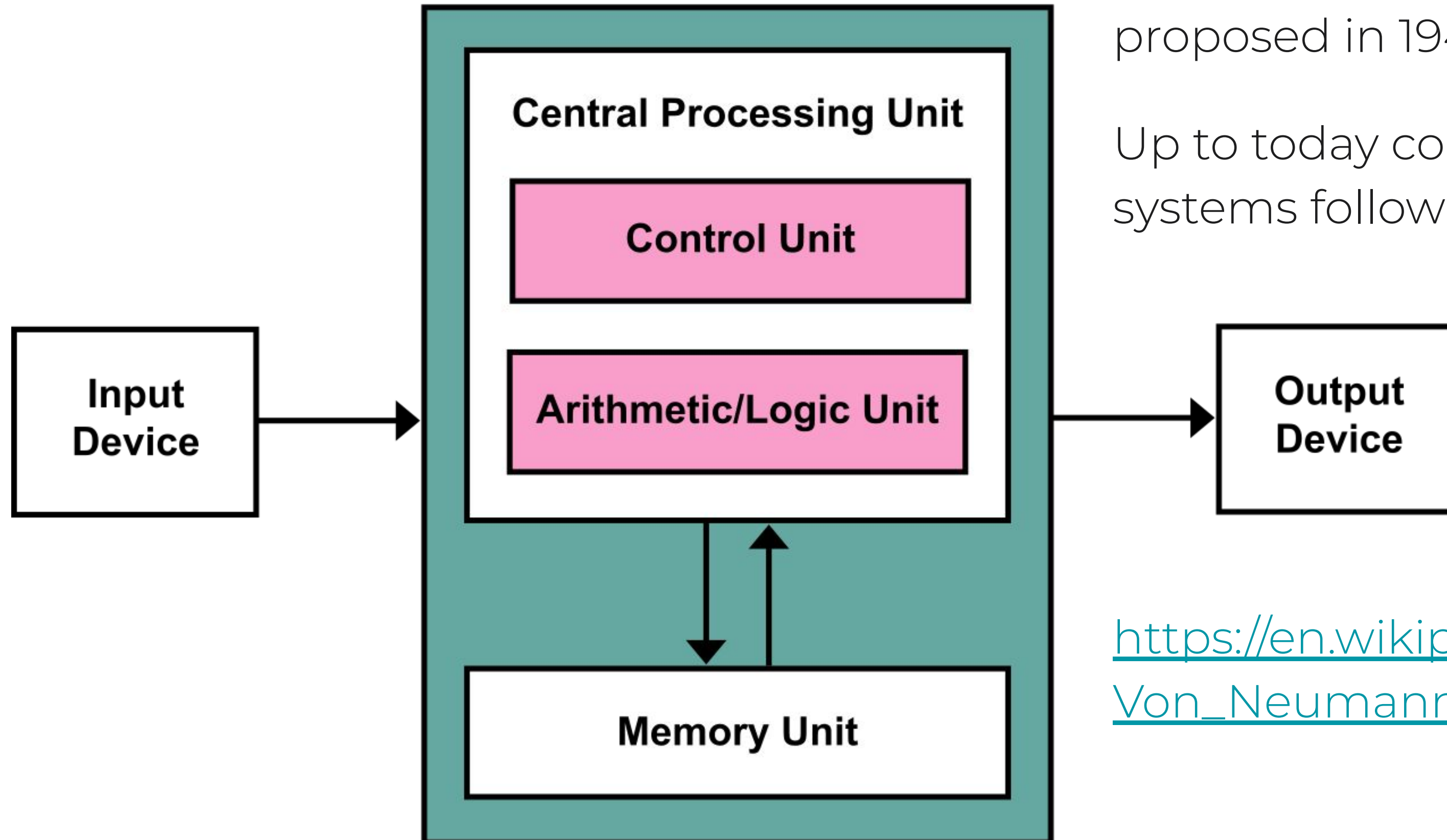
girafe
ai

01

von Neumann architecture

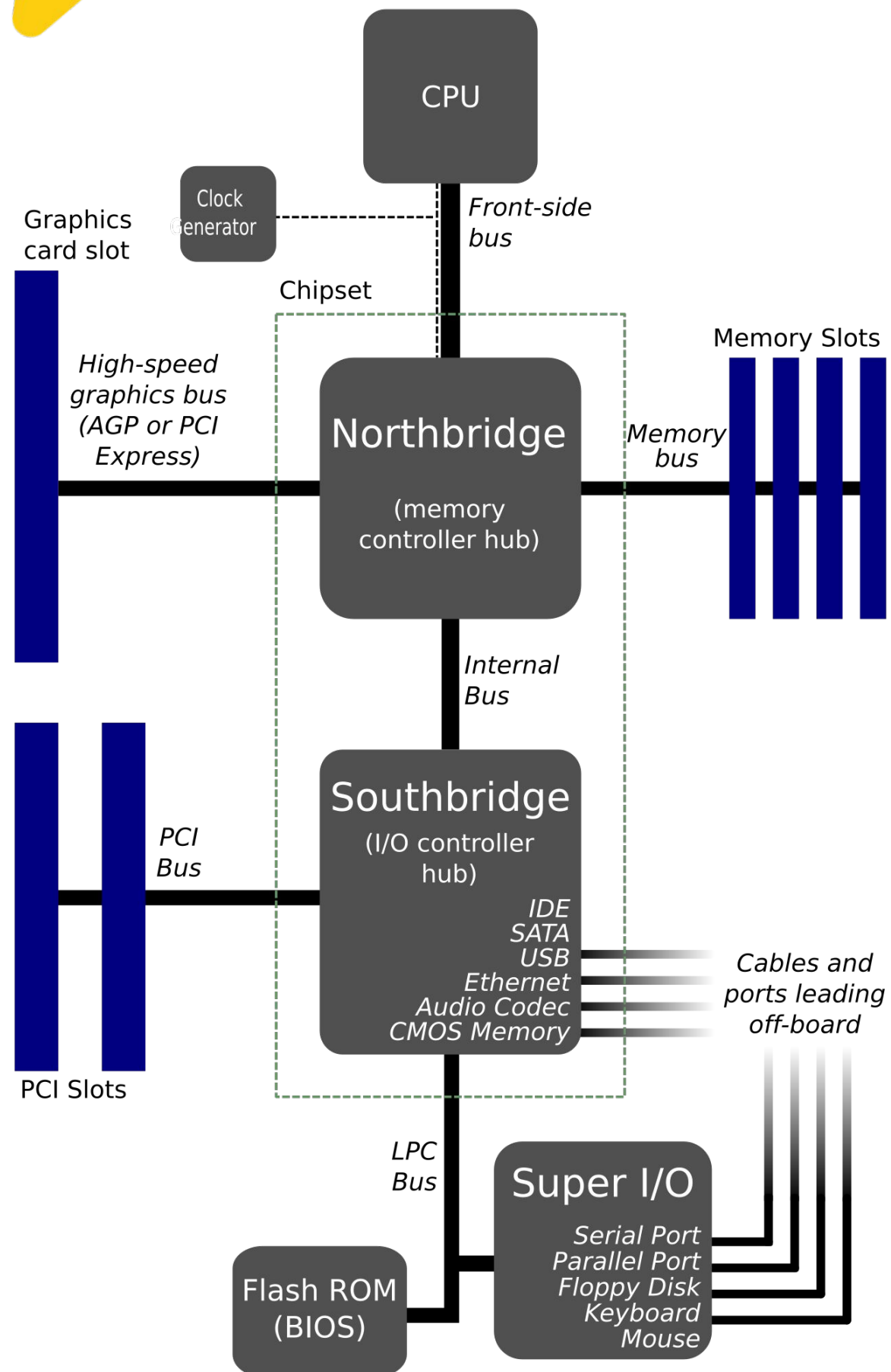
Computer architecture
proposed in 1945.

Up to today computer
systems follow this design.



https://en.wikipedia.org/wiki/Von_Neumann_architecture

von Neumann architecture in the wild



e.g. for many years (up to 2010s) this concept was presented in many motherboards by two chips: [Northbridge](#) (CPU, GPU and memory) [Southbridge](#) (all IO: PCI, SATA, Ethernet, USB)



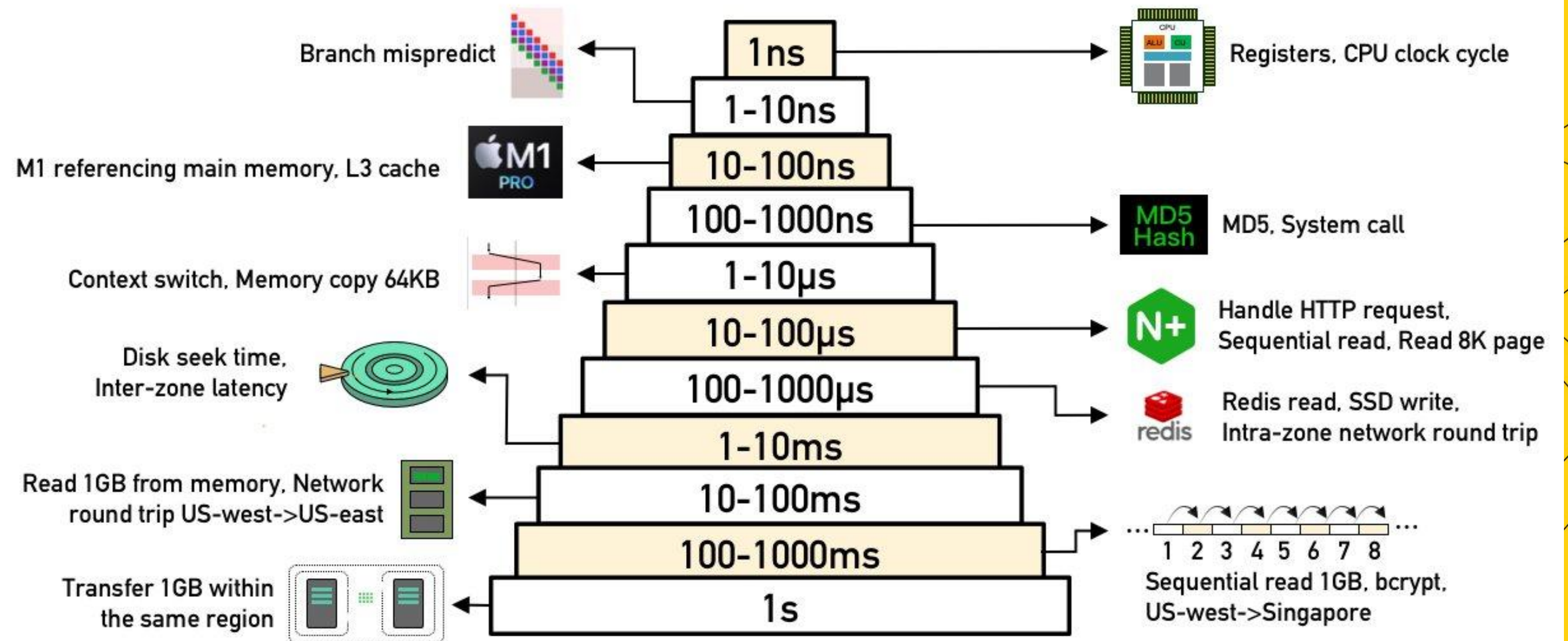
Data Locality principle

[Ulrich Drepper, What every programmer should know about memory \(2007\)](#)

[Stackoverflow discussion](#)

[Locality of reference](#)

[Memory hierarchy](#)

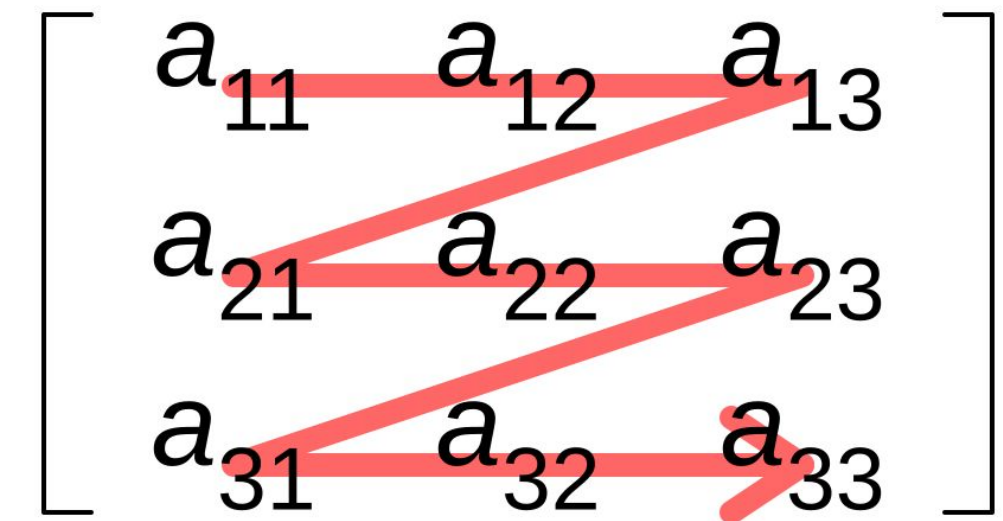


Latency for memory operations in 2020s by bytebytego.com

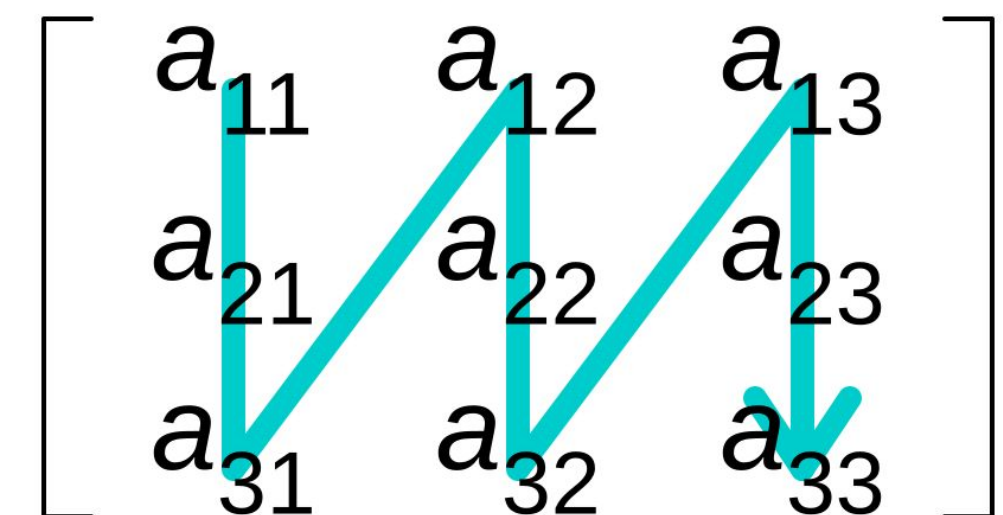
Data Locality examples

- [Why does the order of the loops affect performance when iterating over a 2D array?](#)
- [Loop interchange](#)
- [Caches](#)
- [CPU spatial cache locality in array iteration](#)

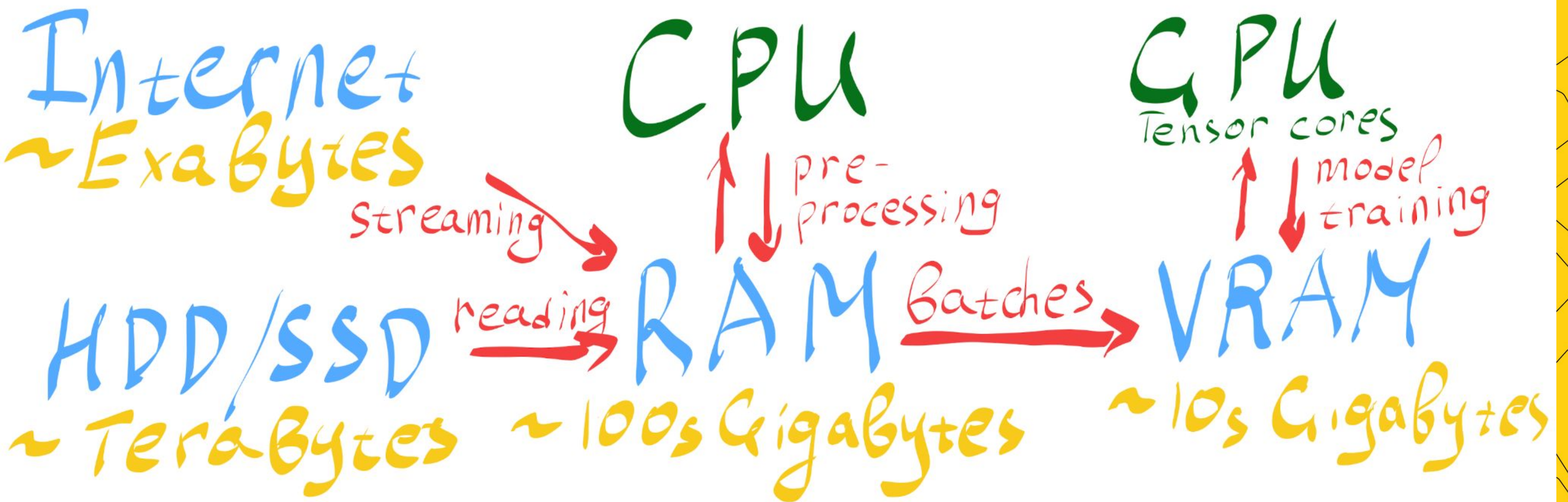
Row-major order



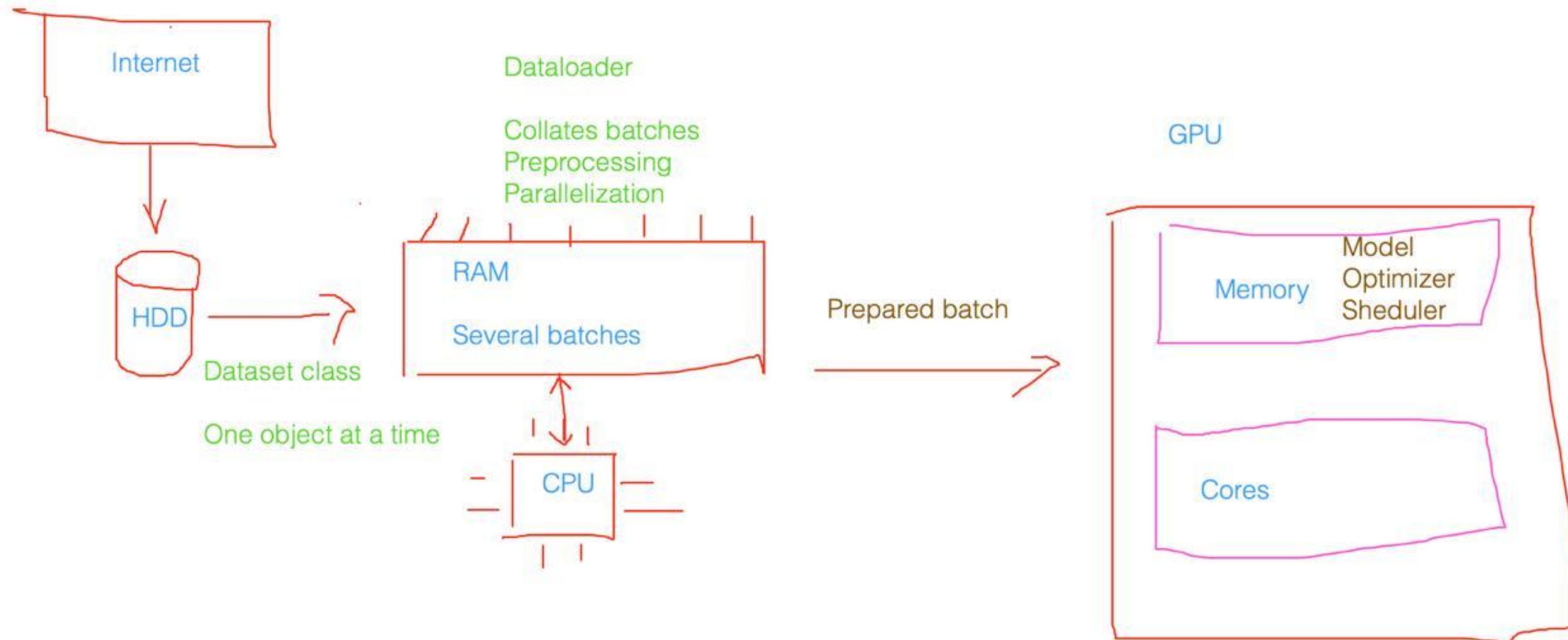
Column-major order



Data Flow in GPU intensive computing



Data Flow in PyTorch



1. Data streaming from network
2. Multi GPU (DDP)
3. GPU only compute (DALLE)

First noted by Gordon Moore in 1965 in his Cramming more components onto integrated circuits article.



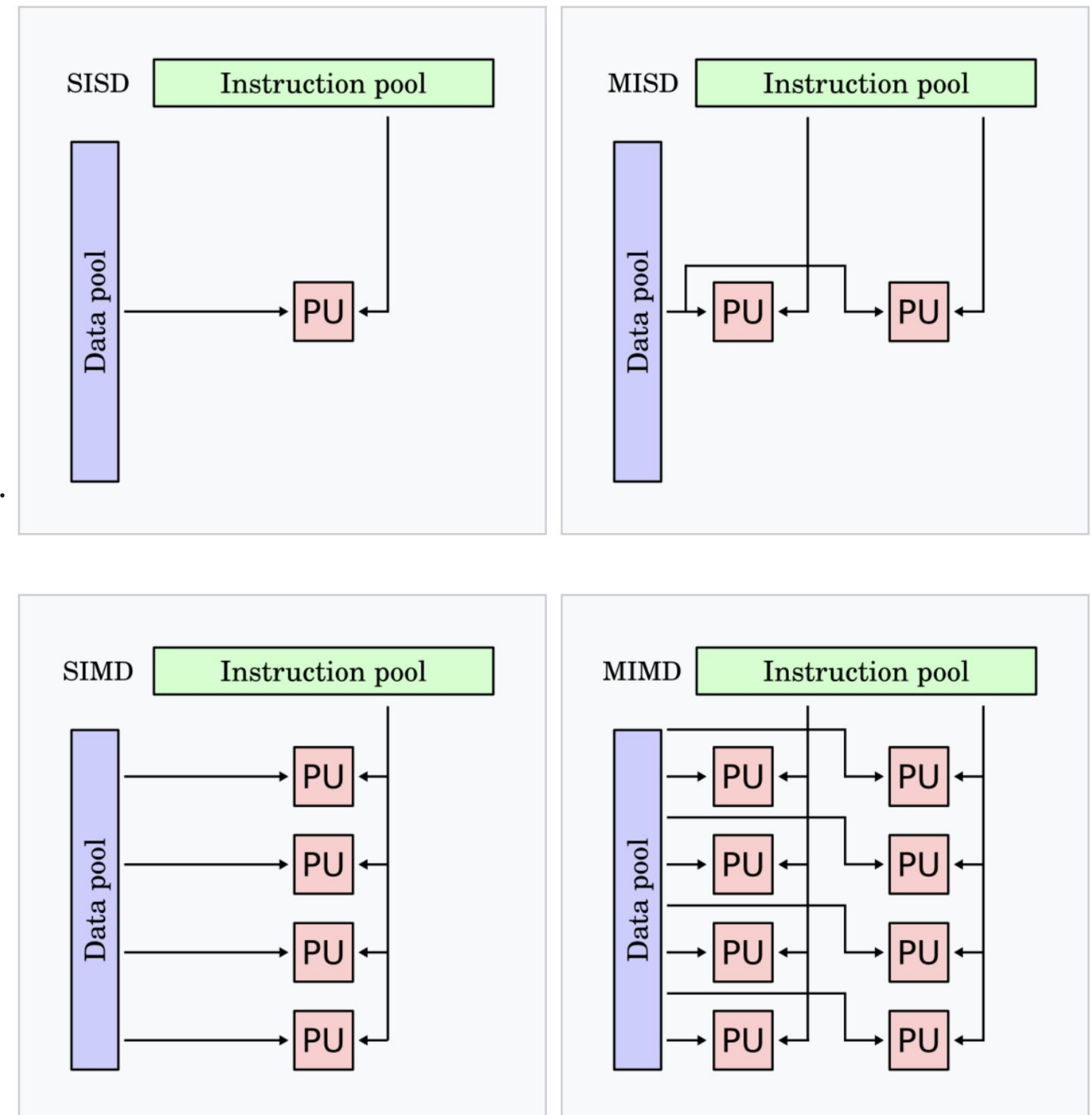
Flynn's taxonomy

classification of multiprocessor computer architectures, proposed by Michael Flynn in 1966.

It has been used as a tool in the design of modern processors and their functionalities.

Further development of this taxonomy leads to pipelined and vector processors.

https://en.wikipedia.org/wiki/Flynn's_taxonomy



Graphical Processing Unit

Brook for GPUs: Stream Computing on Graphics Hardware, 2004

<https://doi.org/10.1145/1015706.1015800>

<https://blogs.nvidia.com/blog/author/ian-buck/>

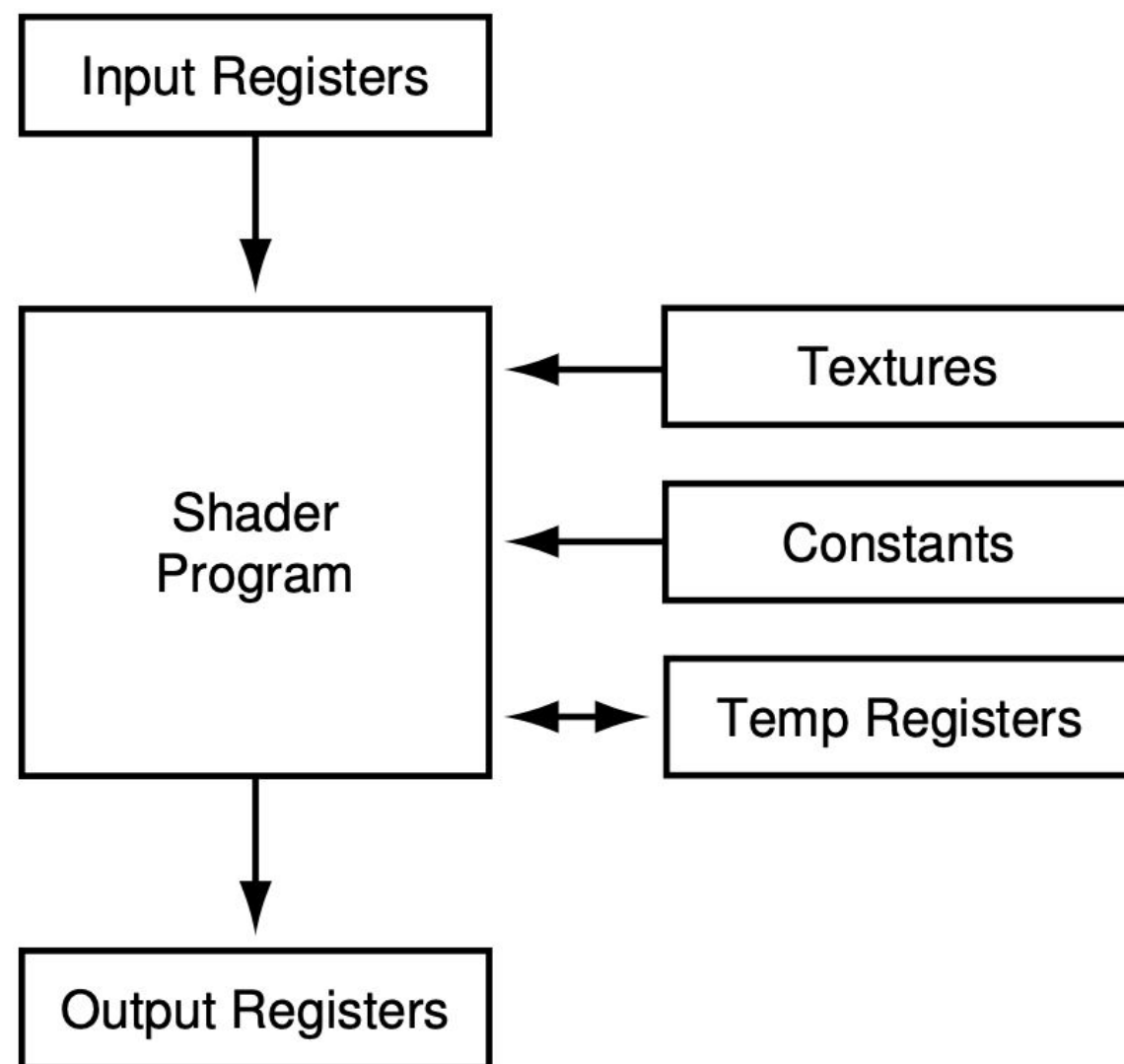


Figure 1: Programming model for current programmable graphics hardware. A shader program operates on a single input element (vertex or fragment) stored in the input registers and writes the execution result into the output registers.



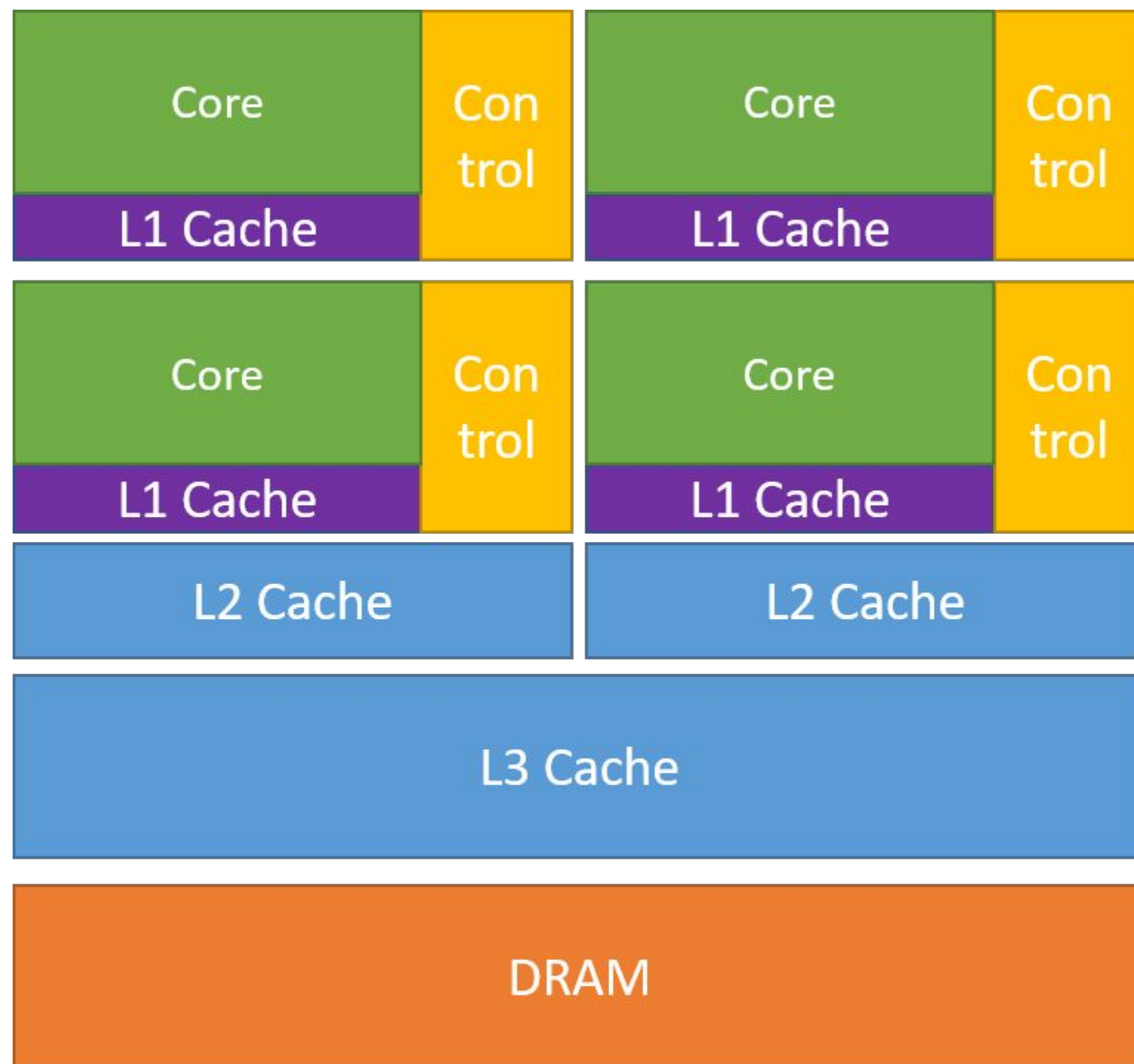
CPU vs GPU



Great hardware overview

<https://www.youtube.com/watch?v=h9Z4oGN89MU>

CPU vs GPU at the lower level



Conclusion on CPU vs GPU

A **CPU** spends silicon making a **small number of complicated fast threads**:

big caches, deep control, branch prediction, latency reduction.

CPU often wins for tiny tensors, tiny batches, lots of Python control flow, or workloads dominated by data movement rather than arithmetic.

A **GPU** spends silicon making a **huge number of simpler threads run together**:

many SMs, many arithmetic units, very high bandwidth, and enough on-chip memory to coordinate massively parallel work.

GPU wins when there is enough parallel work to amortize transfer and launch overhead.

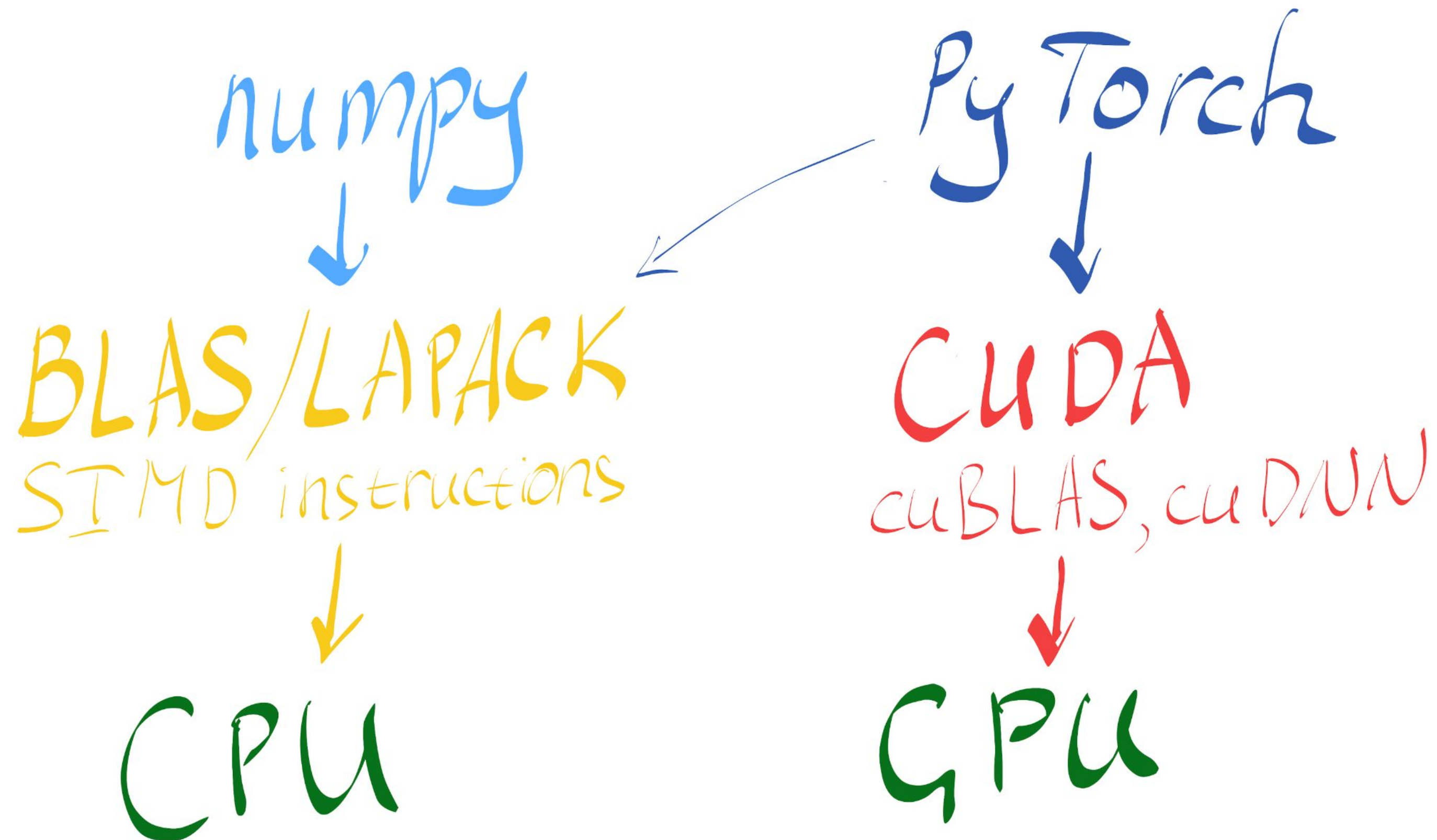
GPU devotes more transistors to data processing,
while the CPU devotes more to caching and flow control.

Hardware comparison numbers

EPYC 9965 datasheet	H100 SXM5 datasheet
192 cores	132 Streaming Multiprocessors include: 16,896 CUDA cores 528 Tensor cores
384 hardware threads	270,000 resident CUDA threads
1 KB registers 15 MB L1 cache 192 MB L2 cache 384 MB L3 cache	33,792 KB registers 30 MB L1 cache (shared per SM) 50 MB L2 cache -
up to 6 TB DDR5 RAM	80 GB HBM3
614 GB/s memory bandwidth	3.3 TB/s memory bandwidth

* All numbers are approximate and rudely unified in sake of contrast

Matrix multiplication infrastructure



CUDA ecosystem

Numpy is concentrated on vectorized processing on CPU. While PyTorch do it on GPU using CUDA.

CUDA which is proprietary parallel computing platform and API that allows to use GPUs for accelerated general-purpose processing.

CUDA API is an extension of the C programming language that adds the ability to specify thread-level parallelism in C and also to specify GPU device specific operations (like moving data between the CPU and the GPU and GPU's virtual instruction set).

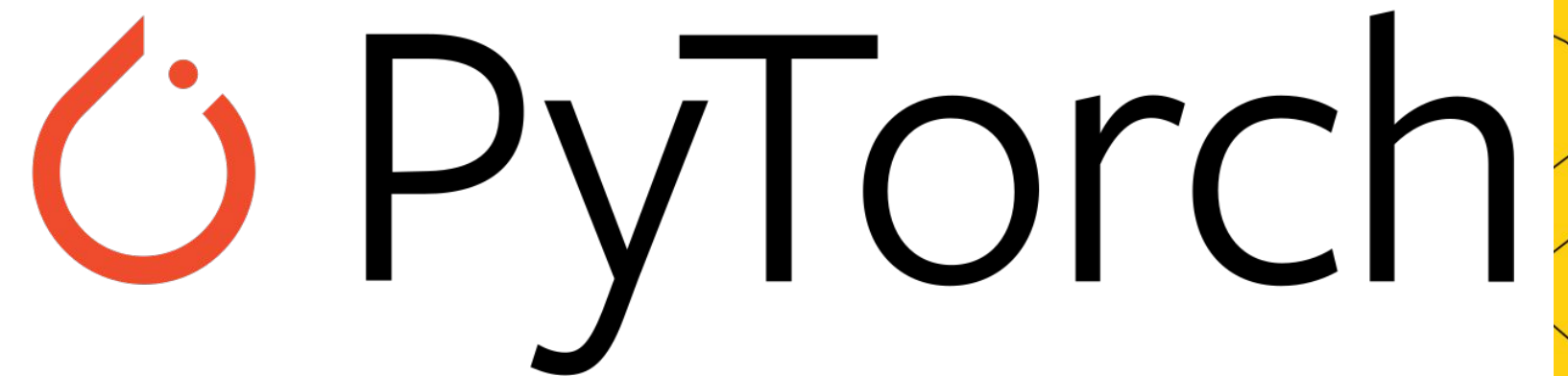
<https://en.wikipedia.org/wiki/CUDA>



Pytorch backends

Main device types in PyTorch:

- cpu
 - like numpy, but slower
- cuda
 - [NVIDIA](#) or [AMD](#) GPUs
 - they share pytorch's semantics
- [mps](#)
 - MacOS GPU cores



Terminology

- GPU is in fact kind of ASIC
 - https://en.wikipedia.org/wiki/Application-specific_integrated_circuit
- term “GPU” first appeared in 70s, but it became common in 1999 with Nvidia marketing campaign
 - https://en.wikipedia.org/wiki/Graphics_processing_unit#Terminology

Contemporary CPUs

- Great overview [ru] <https://www.youtube.com/watch?v=S7xStj7TNjk>
- Architectures (CISC, RISC) [ru] <https://habr.com/ru/companies/selectel/articles/542074/>

Nvidia architectures

girafe
ai

02

Contemporary CPUs

- <https://chatgpt.com/share/69cd1898-a8e4-8389-931f-cb52c56c9cbd>
-

Contemporary GPUs

girafe
ai

03

Types of Nvidia GPU

GPU Computing Applications						
Libraries and Middleware						
cuDNN TensorRT	cuFFT cuBLAS cuRAND cuSPARSE	CULA MAGMA	Thrust NPP	VSIPL SVM OpenCurrent	PhysX OptiX iRay	MATLAB Mathematica
Programming Languages						
C	C++	Fortran	Java Python Wrappers	DirectCompute	Directives (e.g. OpenACC)	
CUDA-Enabled NVIDIA GPUs						
NVIDIA Ampere Architecture (compute capabilities 8.x)					Tesla A Series	
NVIDIA Turing Architecture (compute capabilities 7.x)		GeForce 2000 Series	Quadro RTX Series		Tesla T Series	
NVIDIA Volta Architecture (compute capabilities 7.x)	DRIVE/JETSON AGX Xavier		Quadro GV Series		Tesla V Series	
NVIDIA Pascal Architecture (compute capabilities 6.x)	Tegra X2	GeForce 1000 Series	Quadro P Series		Tesla P Series	
	Embedded	Consumer Desktop/Laptop	Professional Workstation	Data Center		

Double H100 station showcase

<https://www.youtube.com/watch?v=3STJyioJr7k>

Ampere / 30-Series									
	GeForce RTX 3090 Ti	GeForce RTX 3090	GeForce RTX 3080 Ti	GeForce RTX 3080	GeForce RTX 3070 Ti	GeForce RTX 3070	GeForce RTX 3060 Ti	GeForce RTX 3060	GeForce RTX 3050
NVIDIA CUDA Cores	10752	10496	10240	8960 / 8704	6144	5888	4864	3584	2560 / 2304
Boost Clock (GHz)	1.86	1.70	1.67	1.71	1.77	1.73	1.67	1.78	1.78 / 1.76
Memory Size	24 GB	24 GB	12 GB	12 GB / 10 GB	8 GB	8 GB	8 GB	12 GB	8 GB
Memory Type	GDDR6X	GDDR6X	GDDR6X	GDDR6X	GDDR6X	GDDR6	GDDR6	GDDR6	GDDR6
Ada Lovelace / 40-Series									
	GeForce RTX 4090		GeForce RTX 4080		GeForce RTX 4070 Ti				
NVIDIA CUDA Cores	16384		9728		7680				
Boost Clock (GHz)	2.52		2.51		2.61				
Memory Size	24 GB		16 GB		12 GB				
Memory Type	GDDR6X		GDDR6X		GDDR6X				
Max Display Resolution	4K at 240Hz or 8K at 60Hz, with DSC		4K at 240Hz or 8K at 60Hz, with DSC		4K at 240Hz or 8K at 60Hz with DSC, HDR				

Buy or rent?

- Generally owning is cheaper in money, but less stable and scalable
- Decision is highly depend on your goals and budget

At 15% utilization per year, the desktop uses:

$(350 \text{ W (GPU)} + 100 \text{ W (CPU)}) * 0.15 \text{ (utilization)} * 24 \text{ hours} * 365 \text{ days} = 591 \text{ kWh per year}$

So 591 kWh of electricity per year, that is an additional \$71.

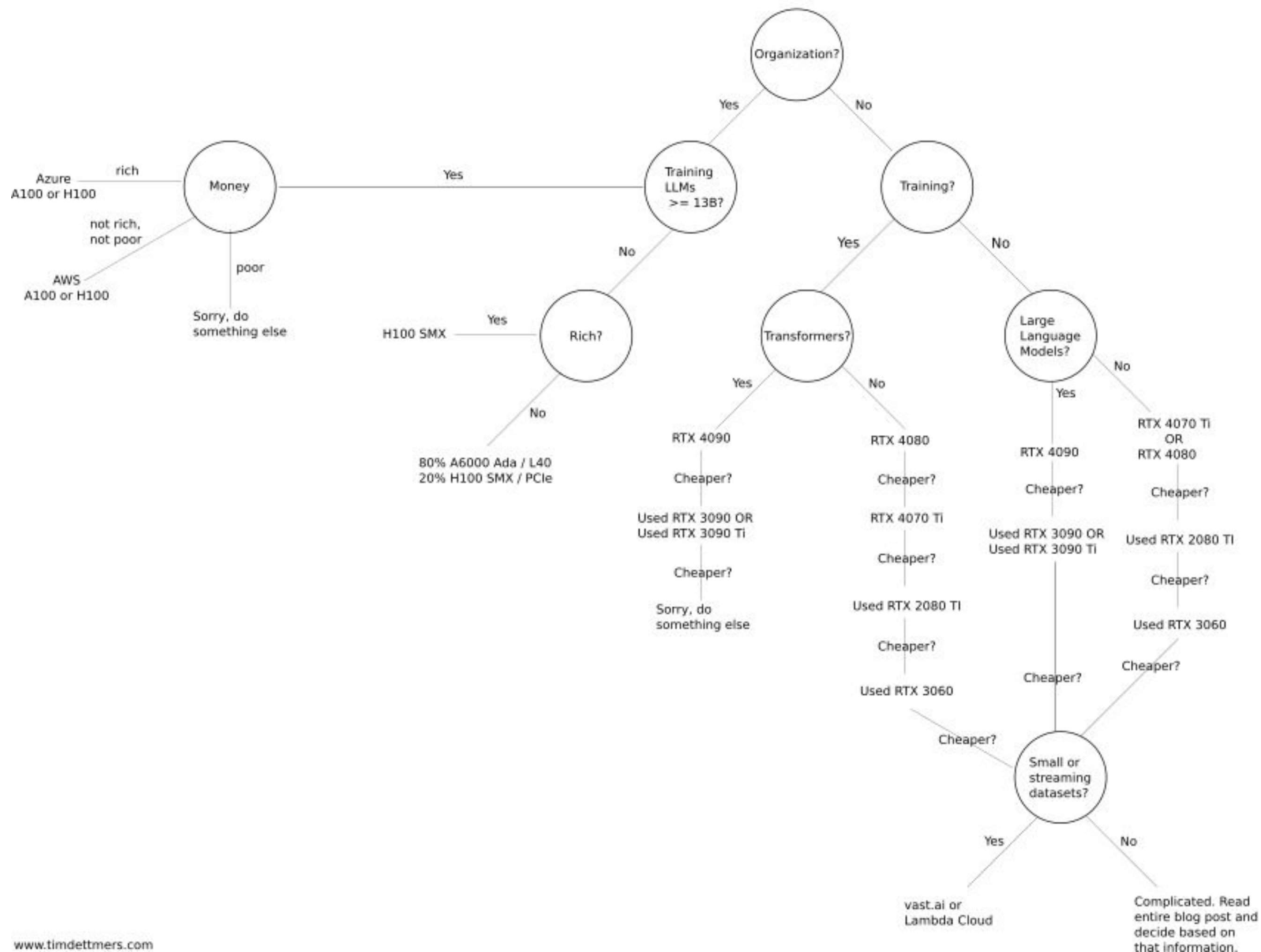
The break-even point for a desktop vs a cloud instance at 15% utilization (you use the cloud instance 15% of time during the day), would be about 300 days (\$2,311 vs \$2,270):

$\$2.14/\text{h} * 0.15 \text{ (utilization)} * 24 \text{ hours} * 300 \text{ days} = \$2,311$

Cloud GPU providers

- Biggest
 - AWS
 - GCS
 - Azure
- Smaller
 - Lambda Cloud
 - [Vast.ai](https://vast.ai)
 - Vultr
 - Rescale

GPU Recommendation Chart



How to choose a GPU

- Comprehensive analysis
 - <https://timdettmers.com/2023/01/30/which-gpu-for-deep-learning/>
- choosing GPU <https://www.youtube.com/watch?v=FlythHjdWI0>
- colab pro overview <https://www.youtube.com/watch?v=ItnX0KDTfJQ>
- one more choose GPU
<https://www.youtube.com/watch?v=YiX9p8A7LqE>
 - <https://docs.google.com/presentation/d/1VZBIqrtOoR2t1Fr-slGJNeMNHLMKvcQD/edit#slide=id.p15>
 - https://docs.google.com/spreadsheets/d/1mD4DrRcq3-y9r86O_P2fD7_0m2f0q9QI/edit?gid=542277665#gid=542277665

LLM inference build

TLDR: VRAM matters the most

https://www.youtube.com/watch?v=P-Fmo_CCIbY

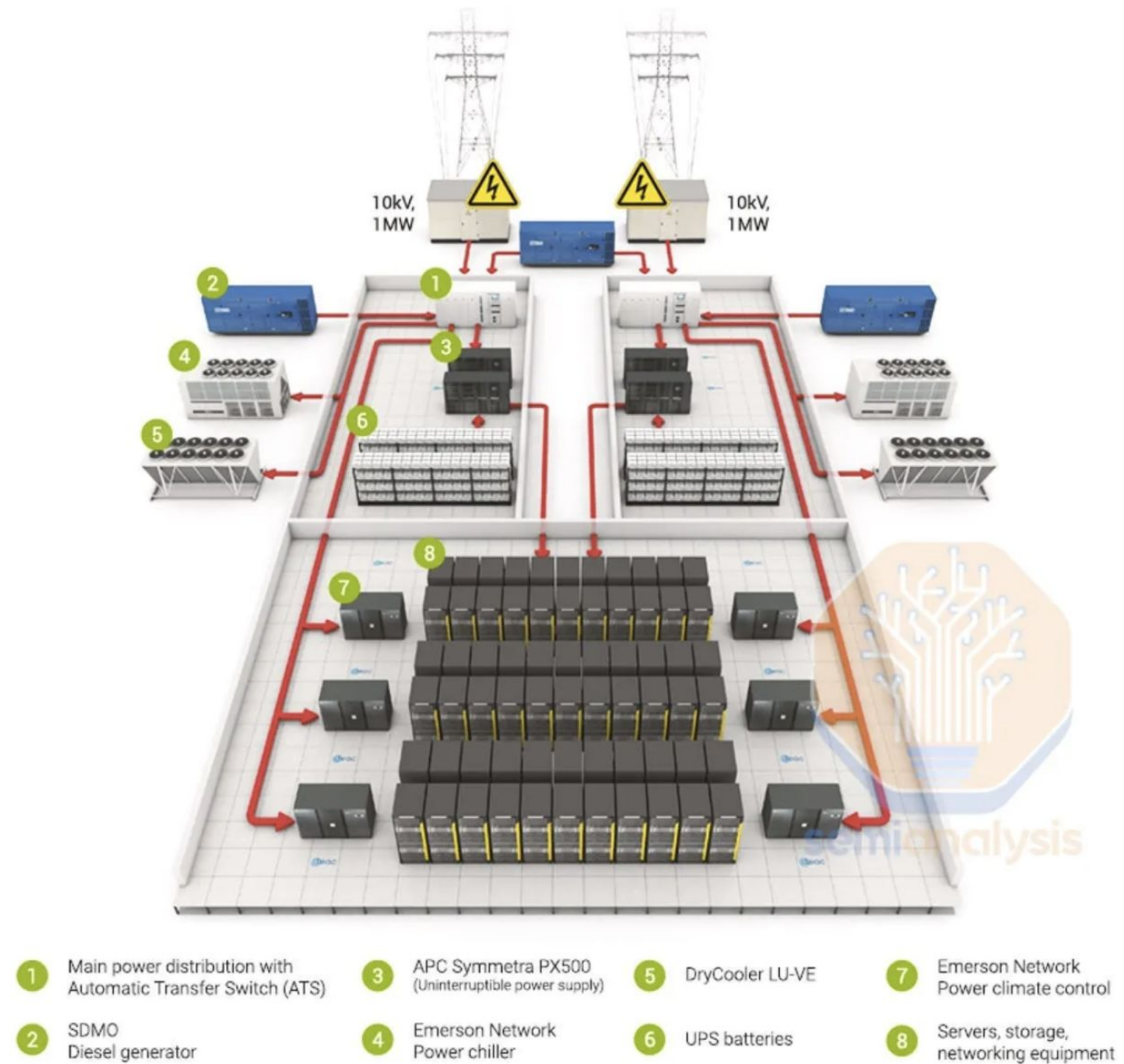
Other manufacturers

- AMD
 - ROCm (Radeon Open Compute Platform) and HIP
- Huawei
 - Ascend
- <https://tenstorrent.com/en/hardware/blackhole>
 - [ru] <https://www.youtube.com/watch?v=pIS3Yery4l0>

Future of datacenters

Now they evolve to fit to GPU
first setup changing power,
latency, availability capacities.

<https://www.youtube.com/watch?v=dhqoTku-HAA>



CUDA

girafe
ai

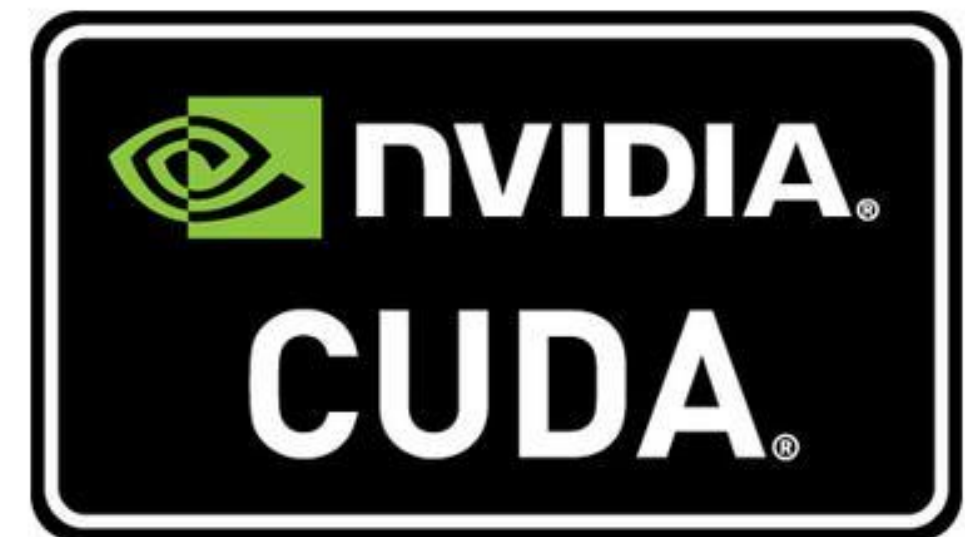
04

Compute Unified Device Architecture

CUDA is a proprietary parallel computing platform and application programming interface (API) that allows software to use certain types of graphics processing units (GPUs) for accelerated general-purpose processing, an approach called general-purpose computing on GPUs. CUDA was created by Nvidia in 2006.

[Wikipedia](#)

You may think of it as an “assembly language” for NVIDIA GPUs.



CUDA cores and Tensor cores

They are just sets of transistor supporting CUDA instructions.
Tensor cores are made for AI and support mixed precision.

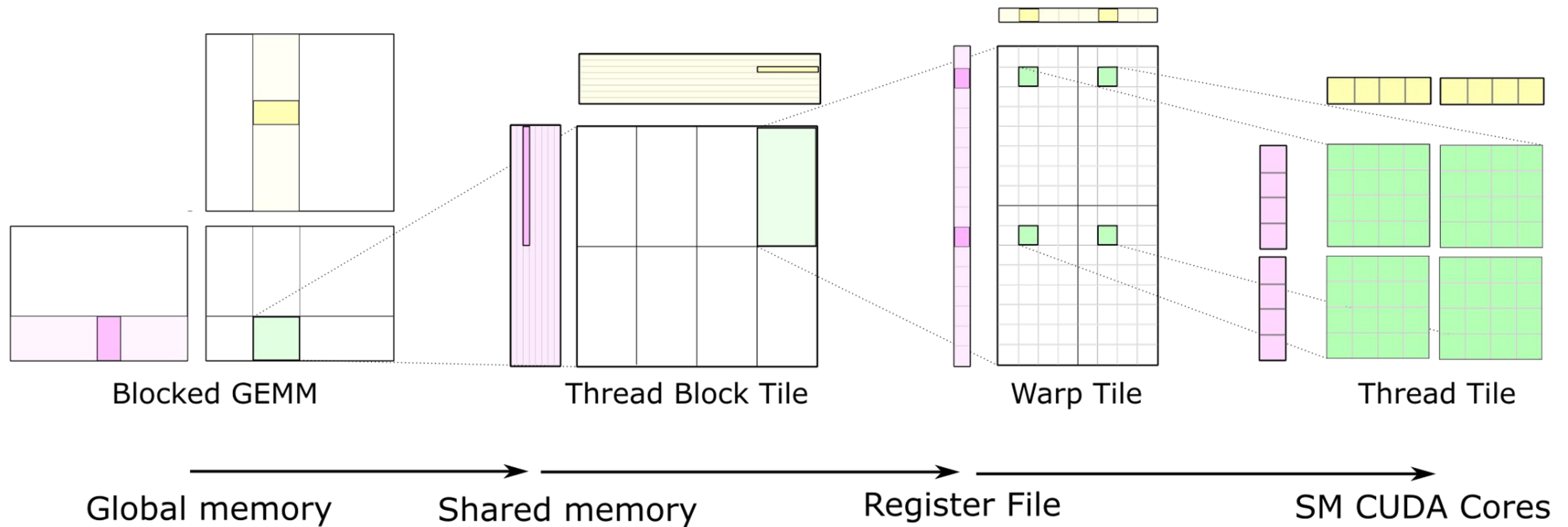
<https://www.youtube.com/watch?v=JFhG9UntZs4>

<https://www.youtube.com/watch?v=fqQyoplWN-8>

Matrix multiplication

<https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html>

Block matrix multiplication



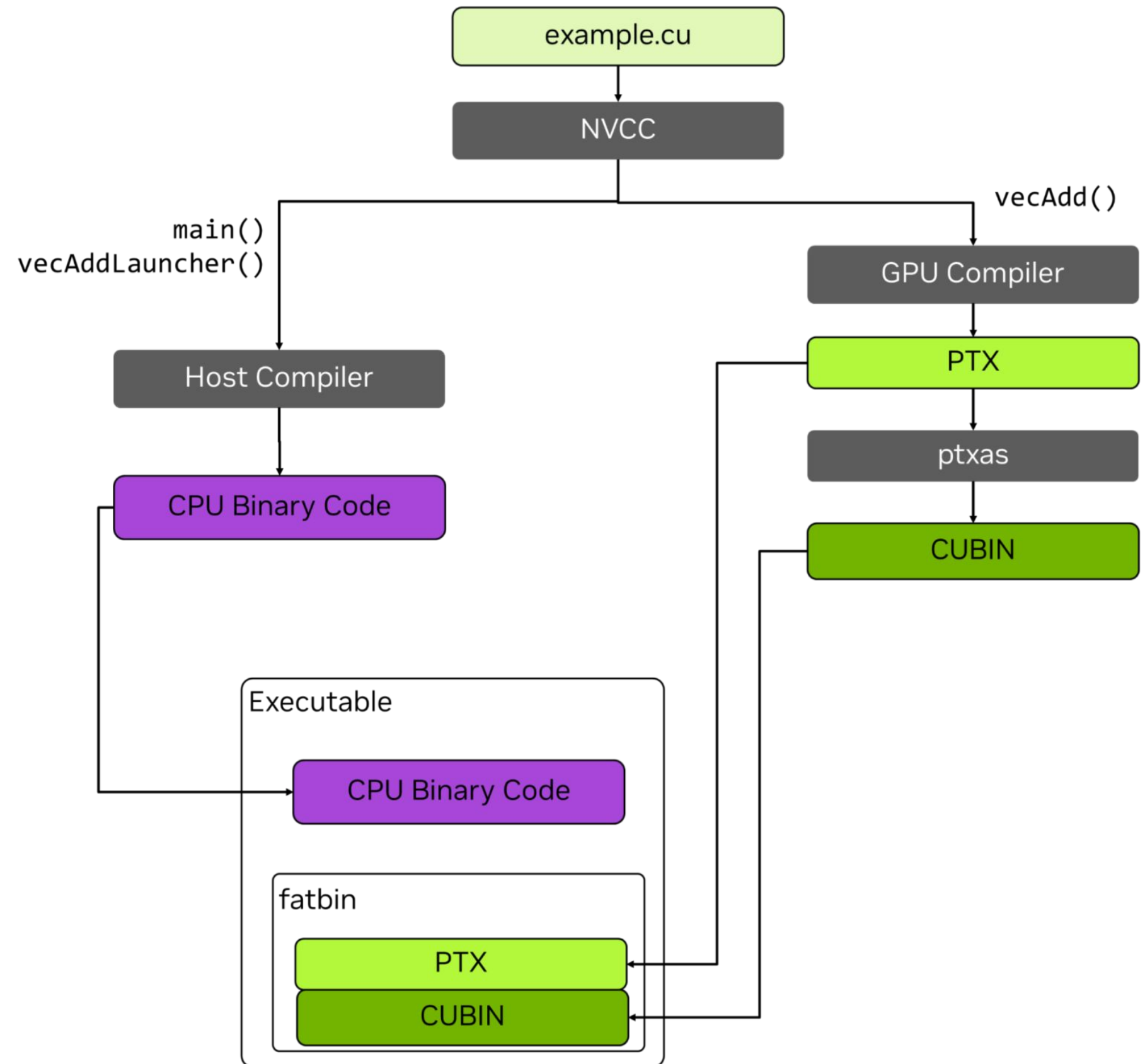
<https://developer.nvidia.com/blog/cutlass-linear-algebra-cuda/>

Deep Learning performance

<https://docs.nvidia.com/deeplearning/performance/index.html>

CUDA compilation

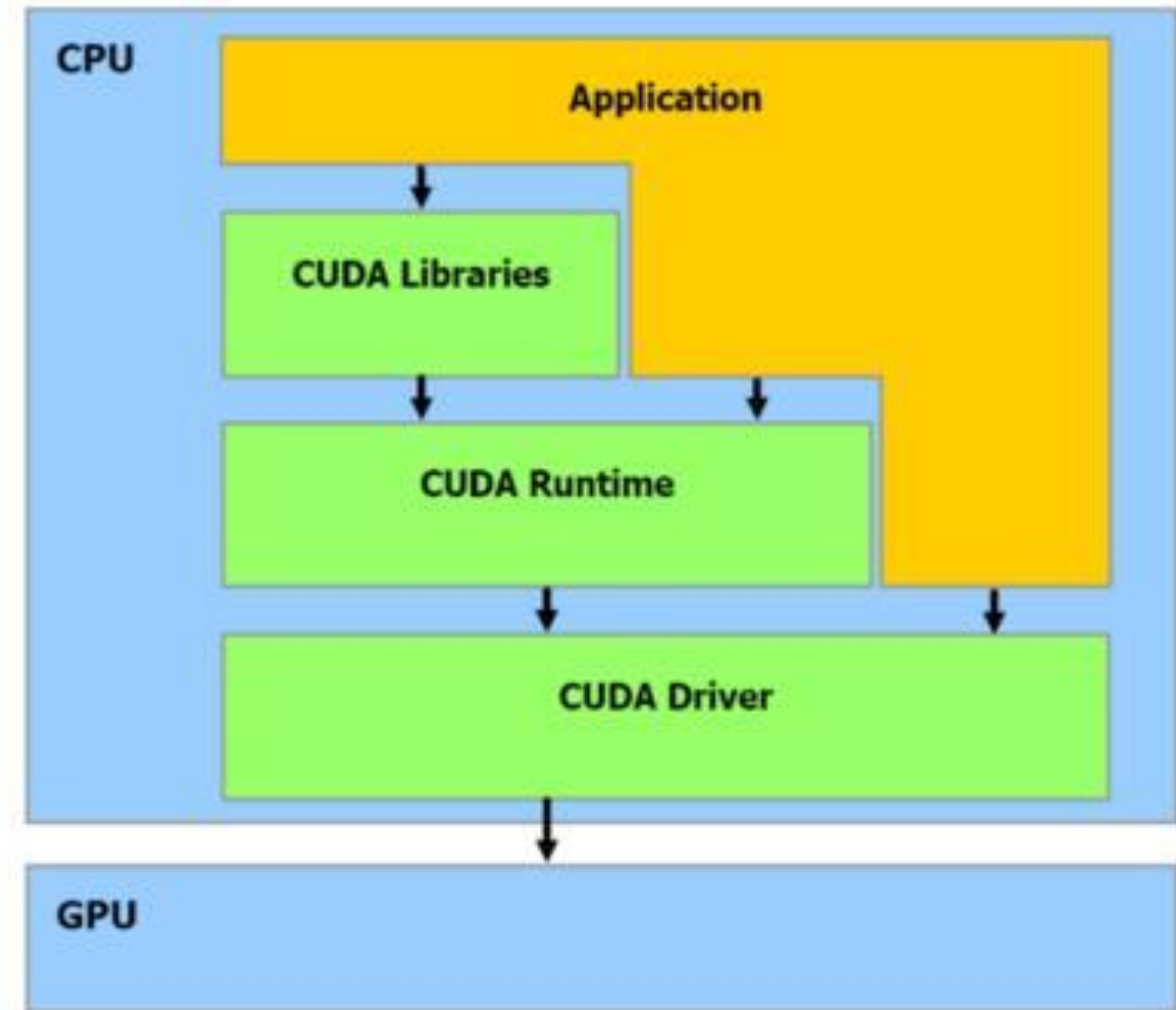
<https://developer.nvidia.com/blog/understanding-ptx-the-assembly-language-of-cuda-gpu-computing/>



CUDA software suite

- Driver
- Runtime
- Libraries
 - cuDNN
 - cuBLAS
 - etc.

[more info](#)



Commands for monitoring

nvidia-smi

<https://github.com/Syllo/nvtop>

<https://developer.nvidia.com/dcgm>

CUDA in Docker

Use official ones!

<https://catalog.ngc.nvidia.com/orgs/nvidia/containers/cuda>

<https://docs.nvidia.com/deeplearning/frameworks/user-guide/index.html>

<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>

<https://nvidia.github.io/container-wiki/toolkit/container-images.html>

Splitting GPU may be painful

There are a few opportunities to do that:

- <https://github.com/Project-HAMi/HAMi>
- vmware

More links

- Choose GPU
 - <https://www.youtube.com/watch?v=H3AQnlpxk0c&t=107s>
 - <https://www.youtube.com/watch?v=ykDuoq-MpHg>
- CUDA course
 - <https://github.com/Infatoshi/cuda-course>
 - https://www.youtube.com/watch?v=86FAWCzle_4
-

Revise

1. Problem statement
2. Metrics
3. Models

Thanks for attention!

Questions?

